Classification and regression: writing our own neural network code

Synne Hjertager Osenbroch, Tora Elise Jaros Madsen, and William Finsnes Naper
Department of Physics, University of Oslo, Norway
(Dated: November 9, 2026)

Neural networks learn their own features from the raw input, while linear models rely on features chosen in advance. To compare the two approaches, a feed-forward neural network (FFNN) with backpropagation was implemented from scratch. The network was applied to regression on the Runge function and to classification of handwritten digits from the MNIST data set. The effects of different optimizers, architectures, learning rates and regularization strengths were investigated. Models and hyperparameters were selected on validation data and the final performance was evaluated on separate test sets. For Runge regression, Adam gave the lowest validation error of the three optimizers plain gradient descent, RMSprop and Adam. The selected network reached a test mean squared error (MSE) of 0.0128, compared with 0.0187 for ordinary least squares (OLS) regression with polynomial degree 8. Among the activation functions, sigmoid gave the lowest validation MSE for the Runge function. Neither L_1 nor L_2 regularization improved the validation MSE. For MNIST, the selected network, with two hidden layers of 100 nodes, Leaky ReLU and L_2 regularization, reached a test accuracy of 97.77 %, compared with 92.53 % for logistic regression. The most frequent misclassification was a 4 predicted as a 9. For the smooth one-dimensional Runge function, the network improved only moderately on polynomial regression, with a test MSE close to the noise level of 0.010. For MNIST, where the relevant features are not known in advance, the network clearly outperformed logistic regression.

I. INTRODUCTION

Neural networks (NNs) are machine learning models inspired by how the brain processes information[1,2]. A feed-forward neural network (FFNN) passes the input through one or more hidden layers of artificial neurons to an output layer. Each neuron computes a weighted sum of its inputs, adds a bias and applies an activation function. During training, the weights and biases are adjusted to reduce a cost function, with the gradients computed by backpropagation. FFNNs are among the most important models in practical machine learning and form the basis of many commercial applications. One example is object recognition in images, which uses convolutional networks, a specialized form of FFNN[2] (Chap. 6).

The non-linear activation functions allow an NN to learn its own features directly from the data. A linear model relies on features chosen in advance. In a previous study (Project 1), the noisy one-dimensional Runge function was fitted with polynomial regression using OLS, Ridge and Lasso[3]. Such a model can miss non-linear structures that the chosen features do not represent. How well an NN performs depends on its architecture, the training and the data and is measured on data that the network has not seen during training[4].

The aim of this project is to implement an FFNN from scratch and study its performance on regression and classification problems. For regression, the network approximates the same Runge function and the results are compared with OLS, Ridge and Lasso from Project 1. For classification, the network is trained to recognize handwritten digits from the MNIST data set and the results are compared with logistic regression. The comparisons are used to assess whether the flexibility of the network justifies its extra cost compared with these simpler models. The project covers the effect of the number of hidden layers and neurons, the activation function, the optimizer, the learning rate and the L_1 and L_2 regularization.

The relevant theory is presented in Sec. II and the implementation and experimental setup are described in Sec. III. The results are presented in Sec. IV and discussed in Sec. V. The conclusions are presented in Sec. VI. All code and results are available at `https://github.com/synneosen/Project-2`.

II. THEORY

A. Regression and classification

Regression and classification are supervised learning tasks. Regression predicts continuous values, while classification assigns each sample to one of a finite set of classes. In this project, regression is used to approximate the Runge function and classification is used to recognize handwritten digits from the MNIST data set[5] (Chap. 8).

B. Feed-forward neural network

A neural network consists of interconnected neurons organized into layers. A single neuron receives multiple input values, multiplies each by a weight, sums the contributions and adds a bias. An activation function is then applied to this sum:

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right) = f(z), \qquad z = \sum_{i=1}^{n} w_i x_i + b. \quad (1)$$

Here, f is the activation function, described in Sec. II C. The quantities x_i, w_i and b denote the input values,

weights and bias, while n is the number of inputs to the neuron. Each weight determines the magnitude and sign of the contribution of its input to the sum z. The bias shifts z independently of the input values. The weights and biases are adjusted during training to reduce the cost function and improve the predictions.

In a feed-forward neural network, the outputs of one layer serve as inputs to the next layer. Information flows from the input layer through the hidden layers to the output layer. This process is called forward propagation. The input layer receives the features, the hidden layers transform them and the output layer produces the predictions. The depth is defined as the number of hidden layers and the width of a hidden layer as its number of neurons[5] (Chap. 8).

C. Activation functions

An activation function is applied to the weighted sum z in each neuron. Non-linear activation functions in the hidden layers allow the network to model non-linear relationships. If all activation functions are linear, the layers combine into a single affine function and the network can only represent a linear model. The derivatives of the activation functions enter the gradients with respect to the weights and biases.

Common choices for the hidden layers are the sigmoid, ReLU and Leaky ReLU functions, described below. The activation function of the output layer is chosen by the type of target: the identity function for regression and the softmax function for multiclass classification.

1. *Sigmoid*

The sigmoid function, also called the logistic function, is defined as

$$f(z) = \frac{1}{1+e^{-z}}. \tag{2}$$

Its derivative is

$$f'(z) = f(z)\big(1-f(z)\big), \tag{3}$$

see Appendix A for the derivation.

The output of the sigmoid lies in the interval $(0,1)$. The derivative has its maximum value of $1/4$ at $z = 0$ and approaches zero for large positive or negative z. During backpropagation, the gradient is multiplied by $f'(z) \leq 1/4$ in each hidden layer. The gradients therefore shrink as they pass through several layers, which slows down learning in the first layers. This is known as the vanishing gradient problem[5] (Chap. 8).

2. *ReLU*

The ReLU function is defined as

$$f(z) = \max(0,z) = \begin{cases} z, & z > 0, \\ 0, & z \leq 0. \end{cases} \tag{4}$$

Its derivative is

$$f'(z) = \begin{cases} 1, & z > 0, \\ 0, & z \leq 0. \end{cases} \tag{5}$$

The derivative is not defined at $z = 0$. The value 0 is used there by convention.

ReLU leaves positive input unchanged and maps negative input to zero. For positive input, the derivative is 1, so the gradient does not shrink through the layers. For negative input, the gradient is zero and the neuron receives no update. If the input stays negative for all samples, the neuron never recovers. This is known as the dying ReLU problem[5] (Chap. 8).

3. *Leaky ReLU*

The Leaky ReLU function is defined as

$$f(z) = \begin{cases} z, & z > 0, \\ \alpha z, & z \leq 0, \end{cases} \tag{6}$$

where α is a small positive constant. Its derivative is

$$f'(z) = \begin{cases} 1, & z > 0, \\ \alpha, & z \leq 0, \end{cases} \tag{7}$$

with the value α used at $z = 0$ by convention. For negative input, the gradient is α instead of zero. The weights of the neuron are still updated, which prevents the dying ReLU problem[5] (Chap. 8).

D. Cost functions

The cost function measures the difference between the predictions of a model and the true values. Let n denote the number of samples, y_i the true data and $\hat{y}_i$ the prediction.

In regression, the cost function is the mean squared error (MSE),

$$C_{\text{MSE}} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2. \tag{8}$$

The network is trained by minimizing the cost function with respect to its weights and biases. The chain rule gives the gradient with respect to a weight w_j,

$$\frac{\partial C_{\text{MSE}}}{\partial w_j} = \frac{2}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)\frac{\partial \hat{y}_i}{\partial w_j}, \tag{9}$$

where

$$\frac{\partial C_{\text{MSE}}}{\partial \hat{y}_i} = \frac{2}{n}(\hat{y}_i - y_i). \tag{10}$$

In multiclass classification, each sample belongs to one of K classes. The targets are one-hot encoded: $y_{ik} = 1$ if sample i belongs to class k and $y_{ik} = 0$ otherwise. The softmax function converts the outputs of the final layer to probabilities,

$$\hat{y}_{ik} = \frac{e^{z_{ik}}}{\sum_{r=1}^{K} e^{z_{ir}}}. \tag{11}$$

The cost function is the multiclass cross entropy (CE),

$$C_{\text{CE}} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} y_{ik}\ln(\hat{y}_{ik}). \tag{12}$$

The derivative with respect to the pre-activation value is

$$\frac{\partial C_{\text{CE}}}{\partial z_{ij}} = \frac{\hat{y}_{ij} - y_{ij}}{n} \tag{13}$$

(Appendix B). The softmax derivative is never needed on its own, since only this combined expression enters backpropagation.

Regularization terms can be added to the cost functions to penalize the weights and reduce overfitting. The terms discussed here are L_1 and L_2,

$$C_{L_1} = C + \lambda\sum_j |w_j|, \qquad C_{L_2} = C + \frac{\lambda}{2}\sum_j w_j^2, \tag{14}$$

where $\lambda \geq 0$ controls the regularization strength. The corresponding weight gradients are

$$\frac{\partial C_{L_1}}{\partial w_j} = \frac{\partial C}{\partial w_j} + \lambda\,\text{sign}(w_j), \qquad w_j \neq 0, \tag{15}$$

$$\frac{\partial C_{L_2}}{\partial w_j} = \frac{\partial C}{\partial w_j} + \lambda w_j. \tag{16}$$

At $w_j = 0$, L_1 uses a subgradient from $[-\lambda, \lambda]$, commonly chosen as zero. The biases are not regularized[5] (Chap. 8).

E. Backpropagation

Backpropagation computes gradients by applying the chain rule backwards through the network[5] (Chap. 8),[6]. Let L denote the output layer and define $\delta^l = \partial C/\partial z^l$, where z^l contains the pre-activation values in layer l.

For an element-wise output activation, the error of the output layer is

$$\delta^L = f'(z^L) \odot \frac{\partial C}{\partial a^L}, \tag{17}$$

where a^L is the prediction, $f'(z^L)$ is the derivative of the activation function and $\odot$ denotes element-wise multiplication. For softmax with cross entropy, the combined derivative from the cost functions is used instead.

The error is propagated backwards through the hidden layers,

$$\delta^l = \left(W^{l+1}\delta^{l+1}\right) \odot f'(z^l). \tag{18}$$

The weights connecting layer l to layer $l+1$ determine how each neuron contributes to the next layer.

Since each bias is added directly to its pre-activation value, its gradient is

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l. \tag{19}$$

The gradient of the weight connecting neuron i in layer $l-1$ to neuron j in layer l is

$$\frac{\partial C}{\partial w_{ij}^l} = \delta_j^l a_i^{l-1}. \tag{20}$$

These expressions apply to one sample without regularization. For a mean loss, the gradients are averaged over the samples and any regularization gradients are added afterwards.

F. Optimization

The gradients from backpropagation are used to update the weights and biases. Gradient descent updates all parameters $\boldsymbol{\theta}$ as

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \gamma\nabla_{\boldsymbol{\theta}} C, \tag{21}$$

where $\gamma > 0$ is the learning rate. A small γ gives slow learning, while a large γ can make the training unstable or diverge. Second-order methods such as Newton's method require the Hessian matrix, whose size grows quadratically with the number of parameters. For neural networks, first-order methods based on the gradient are therefore the standard choice[5] (Chap. 4).

Full-batch gradient descent uses the whole training set for each update. Mini-batch stochastic gradient descent uses shuffled subsets of the data, which gives more updates per epoch at the cost of noisier gradient estimates[2] (Chap. 8). RMSprop scales the step for each parameter by a decaying average of past squared gradients. Adam also keeps a decaying average of the gradient itself and corrects both averages for their initialization at zero[5] (Chap. 4),[2] (Chap. 8).

G. Weight initialization

The weights are initialized randomly. Equal weights would give all neurons in a layer the same output and the same gradient, so they would remain identical. Random weights break this symmetry and the biases can be initialized to zero. The variance of the weights is chosen so that the signal is preserved through the layers. With

M_{l-1} inputs to layer l, this requires $\mathrm{Var}(w) = 1/M_{l-1}$ in the forward pass. Xavier initialization, $\mathrm{Var}(w) = 2/(M_{l-1} + M_l)$, also accounts for the backward pass and He initialization, $\mathrm{Var}(w) = 2/M_{l-1}$, compensates for ReLU setting half of its inputs to zero[5] (Chap. 8).

III. METHODS

A. Data and preprocessing

1. Runge function

The data set used in the first part of the project consists of $n = 100$ data points sampled from the one-dimensional Runge function with added Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma^2)$ and $\sigma = 0.1$. The seed 2026 is used for drawing the x-values from a uniform distribution and for drawing the Gaussian noise for each y-value. The input x is standardized to zero mean and unit variance. The target y is centered by subtracting its mean.

The data are split into a training set of 80 points and a test set of 20 points using `train_test_split` from Scikit-Learn[7]. The test set is held out until the final evaluation. The training set is split further into 60 training points and 20 validation points. The validation set is used to select hyperparameters. All statistics are computed on the training subset only. This is to ensure that information from the validation and test data does not enter the model.

2. MNIST

The MNIST data set consists of 70 000 grey-scale images of handwritten digits from 0 to 9, each of size 28×28 pixels[8]. The data set comes with a canonical split into 60 000 training images and 10 000 test images. Each image is flattened to a vector of 784 features. The pixel values are divided by 255 to map them from $[0, 255]$ to $[0, 1]$. The class labels are one-hot encoded as vectors of length 10.

A validation set of 10 000 images was drawn from the training images, leaving 50 000 images for training. The split was stratified, so the class proportions are the same in the training and validation sets and used the seed 2026. The validation set was used for all hyperparameter tuning and model selection and the test set was used once, to evaluate the final models. The classes are nearly balanced. Of the 50 000 training images, the digit 1 is the most frequent class, with 11.2 % and the digit 5 is the least frequent, with 9.0 %. Accuracy is therefore a meaningful performance measure.

Table I summarizes the data sets.

TABLE I. Data sets used in this work.

Data set	Features	Train	Validation	Test
Runge	1	60	20	20
MNIST	784	50 000	10 000	10 000

B. Feed-forward neural network

A feed-forward neural network object was programmed, taking a flexible number of hidden layers, nodes and activation function for each layer. The available activation functions were sigmoid, ReLU, Leaky ReLU, identity and softmax. Identity activation was used in the output layer for regression, since a continuous value is the desired output and softmax was used for MNIST classification, since it gives a probability for each class. For regression, the input was x alone, since the hidden layers form their own non-linear features.

The weights were drawn from $\mathcal{N}(0, 1/M_{l-1})$ for sigmoid, identity and softmax layers and from $\mathcal{N}(0, 2/M_{l-1})$ for ReLU and Leaky ReLU layers, where M_{l-1} is the number of inputs to layer l. The biases were set to zero and the seed 2026 was used for reproducibility. During forward propagation, the values of each layer were stored for use in backpropagation. The gradients of the cost function with respect to each parameter were calculated with the chain rule. MSE was used as cost function for regression and multiclass cross entropy for classification. The gradients were passed to the selected optimizer, which performed the parameter update.

C. Optimization

Plain gradient descent and mini-batch stochastic gradient descent were implemented, together with the adaptive methods RMSprop and Adam. The gradients from backpropagation were used to update the weights and biases, with the update rules from Project 1[3]. The training data were reshuffled at the start of each epoch. RMSprop used $\rho = 0.9$ and $\epsilon = 10^{-8}$ and Adam used $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. In the learning-rate analysis for regression, γ was varied from 10^{-3} to 1. The batch size of each experiment is given in Sec. III D.

D. Experimental setup

1. Learning rate and optimizer

The networks for the Runge function used sigmoid activation in the hidden layers. Two architectures were compared: One hidden layer of 50 nodes and two hidden layers of 100 nodes. Plain gradient descent used full-batch updates, while RMSprop and Adam used mini-batches of 16 samples. Table II lists the hyperparameters. Each

combination was trained for 10000 epochs, with the validation MSE being recorded after each epoch. For each architecture, the optimizer and learning rate with the lowest validation MSE were selected, along with the corresponding epoch. The selected networks were retrained on the 60 training points and evaluated on the test set. The test MSE was compared with OLS on the same data, with the polynomial degree found in Project 1[3] to produce the lowest MSE: degree 8.

TABLE II. Hyperparameters in the learning-rate study for regression.

Hyperparameter	Values
Architecture	$[1, 50, 1]$, $[1, 100, 100, 1]$
Optimizer	GD (full batch), RMSprop and Adam (batch size 16)
γ	10 values, log-spaced from 10^{-3} to 1
Epochs	10000

2. *Activation function and architecture*

The activation functions sigmoid, ReLU and Leaky ReLU were compared for networks with 1, 2 and 5 hidden layers of 15, 50 and 100 nodes each, which gives 27 combinations. All networks were trained for 1000 epochs with Adam, $\gamma = 10^{-3}$ and full-batch updates, without regularization and with the seed 2026. The combination with the lowest validation MSE was trained again on the 60 training points and evaluated on the test set.

3. *Regularization*

The L_1 and L_2 penalties were investigated separately and were applied to the weights only. For each penalty, all combinations of the hyperparameters in Table III were trained, which gives 324 configurations per penalty. All models were trained for 1000 epochs with Adam and full-batch updates, with the seed 2026 for both the initialization and the shuffling, on the data split described in Sec. III A. For each penalty, the configuration with the lowest validation MSE was selected and evaluated on the test set. The test MSEs of the selected L_1 and L_2 networks were compared with those of Ridge and Lasso regression. For both, polynomial degrees from 1 to 15 were tested with 100 logarithmically spaced regularization parameters between 10^{-6} and 1. Ridge was solved analytically and Lasso was solved with Adam, $\gamma = 0.01$ and 10000 iterations. Both models were trained on the 80 training points and the degree and regularization parameter with the lowest test MSE were selected.

TABLE III. Hyperparameters in the regularization study, with the regularization parameter λ and the learning rate γ. All combinations were trained for both the L_1 and the L_2 penalty.

Hyperparameter	Values
λ	0, 10^{-4}, 10^{-3}, 10^{-2}
γ	10^{-4}, 10^{-3}, 10^{-2}
Activation	Sigmoid, ReLU, Leaky ReLU
Hidden layers	1, 2, 5
Nodes	15, 50, 100

4. *Classification*

The MNIST networks used a softmax output and the cross entropy as cost function and were trained with mini-batches of 128 images. The optimizers Adam and RMSprop were first compared for a network with one hidden layer of 100 nodes and ReLU activation, with $\gamma \in \{10^{-4}, 10^{-3}, 10^{-2}\}$ and 30 epochs. With Adam and $\gamma = 10^{-3}$, the activation functions sigmoid, ReLU and Leaky ReLU were compared for 1, 2 and 5 hidden layers of 15, 50 and 100 nodes. For the best architecture, the L_2 penalty was tested with $\lambda \in \{0, 10^{-4}, 10^{-3}, 10^{-2}\}$. All models were selected by the lowest validation loss. The final network was trained on the 50 000 training images for 22 epochs, the epoch with the lowest validation loss and evaluated on the test set.

E. Evaluation metrics

The regression models are evaluated with the MSE on the test data. The classification models are evaluated with the accuracy and the confusion matrix, with the true class on the rows and the predicted class on the columns. Multinomial logistic regression from Scikit-Learn[7], with the saga solver, `max_iter` = 1000 and the default L_2 penalty ($C = 1$), trained on the same 50 000 training images with the seed 2026, serves as the classification baseline.

IV. RESULTS

A. Regression on the Runge function

Figure 1 and Figure 2 show that plain GD performance has a stronger dependence on learning rate than RMSProp and Adam. The network with two hidden layers reaches a validation MSE close to the optimal at low learning rates for RMSProp and Adam, while the one hidden layer network shows a more stable decrease with learning rate for these optimizers. At high learning rates, the validation MSE increased more for two hidden layers than for one.

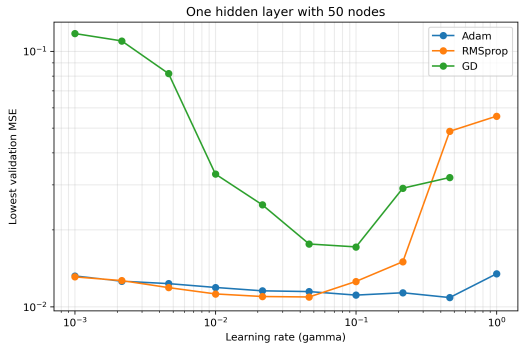


FIG. 1. Lowest validation MSE over 10000 epochs as a function of the learning rate γ for plain gradient descent, RMSprop and Adam, for one hidden layer of 50 nodes with sigmoid activation and an identity output. Single seed.

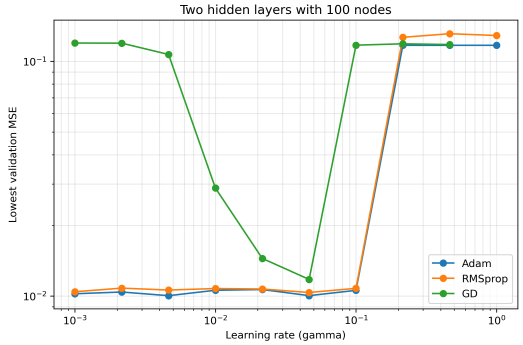


FIG. 2. Lowest validation MSE over 10000 epochs as a function of the learning rate γ for plain gradient descent, RMSprop and Adam, for two hidden layers of 100 nodes with sigmoid activation and an identity output. Single seed.

Adam gave the lowest validation MSE for both architectures, 0.0109 for one hidden layer and 0.0100 for two hidden layers (Table IV). The selected learning rates were 0.46 and 0.0046. Both networks reached a lower test MSE than OLS, 0.0151 and 0.0128 compared with 0.0187.

TABLE IV. Validation and test MSE for the selected FFNN architectures and the polynomial OLS regression model from Project 1. The optimal optimizer and learning rate are shown for each neural network (NN) architecture.

Model	Optimizer	γ	Val. MSE	Test MSE
OLS	–	–	–	0.01871
NN $[1, 50, 1]$	Adam	0.46416	0.01086	0.01511
NN $[1, 100, 100, 1]$	Adam	0.00464	0.01004	0.01280

B. Architectures

Figures 3 and 4 show the training and validation MSE for Sigmoid, ReLU and Leaky ReLU as a function of the number of hidden layers and number of nodes per layer.

For sigmoid with 15 nodes per layer the validation MSE increased with depth. It increased from 0.025 with one hidden layer to 0.101 with five. The training MSE of this network was also high. With 100 nodes per layer the validation MSE for sigmoid decreased with depth, from 0.026 to 0.012. With 50 nodes, it decreased from 0.017 to 0.011 between one and two hidden layers and with five the MSE was 0.012.

For ReLU and Leaky ReLU some of the deepest networks had a low training MSE and a higher validation MSE. ReLU with five hidden layers of 50 nodes had training MSE of 0.011 and a validation MSE of 0.024. Leaky ReLU with five hidden layers of 100 nodes had a training MSE of 0.013 and validation MSE of 0.034.

The lowest validation MSE was 0.011. It was obtained with sigmoid with two hidden layers and 50 nodes per layer. Several other configurations had a validation MSE of 0.012. After retraining, the selected model had a test MSE of 0.014.

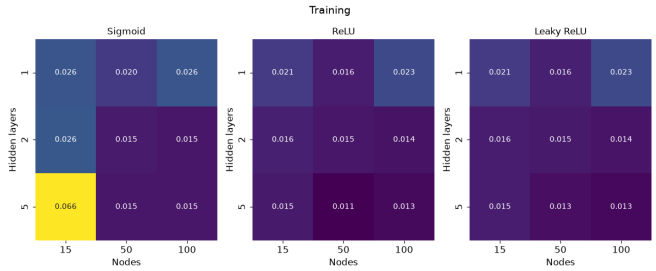


FIG. 3. Training MSE heatmaps as the function of the number of hidden layers and the number of nodes per layer for Sigmoid, ReLU and Leaky ReLU.

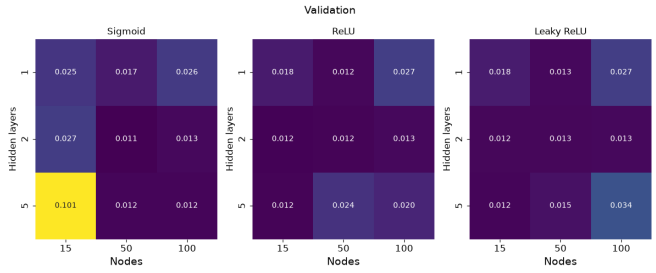


FIG. 4. Validation MSE heatmaps as the function of the number of hidden layers and the number of nodes per layer for Sigmoid, ReLU and Leaky ReLU.

C. Regularization

For L_1 and L_2, architectures and hyperparameters were varied. Sigmoid with two layers, 50 nodes per layer, a learning rate of 0.001 and $\lambda = 0$ obtained the lowest validation MSE for both penalties. It was measured to be 0.011. Finally, the test MSE for both was measured to be 0.0141. The results are summarized in Table V.

Table VI compares the test MSE of the selected FFNN with linear regression using Ridge and Lasso penalties. The selected FFNN achieved the lowest test MSE of the three models.

TABLE V. Selected architectures, hyperparameters and MSE values from the L_1 and L_2 regularization searches. Both searches selected $\lambda = 0$, corresponding to the same unregularized network and therefore identical results.

Quantity	L_1	L_2
Activation function	Sigmoid	Sigmoid
Layers	2	2
Nodes per layer	50	50
γ	0.001	0.001
λ	0	0
Validation MSE	0.0110	0.0110
Test MSE	0.0141	0.0141

TABLE VI. Test MSE for Ridge and Lasso regression from Project 1 and the selected FFNN from the regularization study. Different methods applied: Ridge and Lasso used 80 training points, while FFNN used 60 training points and 20 validation points for hyperparameter optimization.

Model	Degree	λ	Test MSE
Ridge	8	1.87×10^{-5}	0.01766
Lasso	8	2.01×10^{-4}	0.01745
FFNN	–	0	0.01410

D. Classification

Table VII compares Adam and RMSprop at three learning rates. At $\gamma = 0.0001$, both optimizers reached a validation accuracy of 0.96, with a minimum validation loss of 0.1413 for Adam and 0.1443 for RMSprop. At $\gamma = 0.001$ and $\gamma = 0.01$, Adam reached a lower minimum validation loss than RMSprop. The lowest minimum validation loss, 0.0878, was obtained with Adam and $\gamma = 0.001$.

TABLE VII. Validation results for classification of the MNIST data set with Adam and RMSprop for different learning rates γ. The validation accuracy is measured after the final epoch.

Optimizer	γ	Min. val. loss	Best epoch	Final val. accuracy
RMSprop	0.0001	0.1443	30	0.9593
	0.001	0.0939	12	0.9768
	0.01	0.1549	3	0.9736
Adam	0.0001	0.1413	30	0.9596
	0.001	0.0878	14	0.9771
	0.01	0.1178	2	0.9742

The training and validation losses for Adam at different learning rates are shown in Figures 5 and 6, respectively. For $\gamma = 0.0001$, the validation loss was still decreasing at the end of the simulation. At $\gamma = 0.01$, the validation loss increased rapidly after reaching its minimum, while the training loss continued to decrease. At $\gamma = 0.001$, the validation loss started to rise at the end of the simulation.

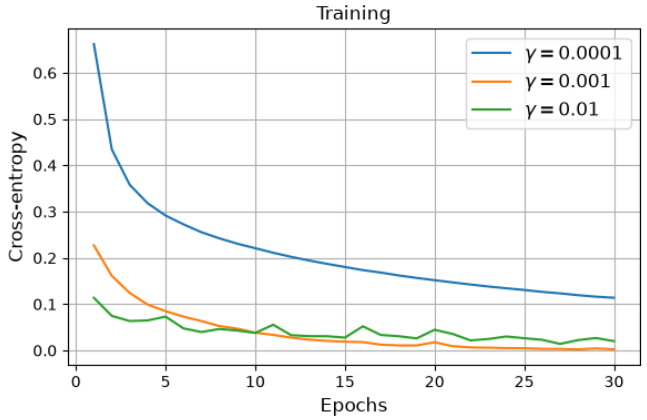


FIG. 5. Training Cross-entropy loss as a function of epochs for the Adam optimizer. Each curve corresponds to a different learning rate, γ.

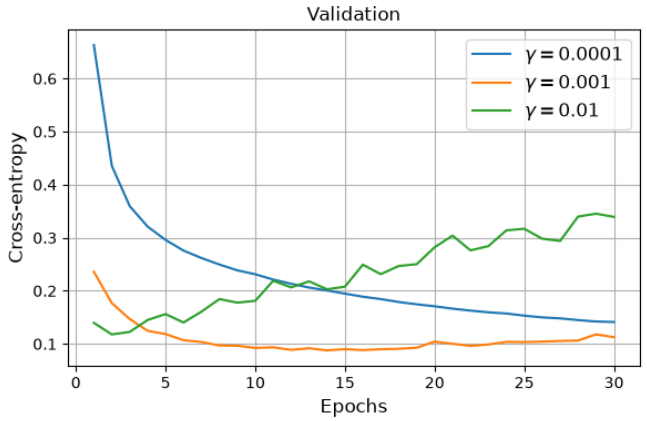


FIG. 6. Validation Cross-entropy loss as a function of epochs for the Adam optimizer. Each curve corresponds to a different learning rate, γ.

Table VIII shows the best architecture for each activation function. The Leaky ReLU obtained the minimal validation loss. It was 0.0875. The minimum was reached after 6 epochs. After this, the training loss continued to decrease while the validation loss increased (Figure 7).

TABLE VIII. Best MNIST architecture for each activation function, selected by minimum validation loss. The validation accuracy is measured after the final epoch.

Activation	Layers	Nodes	Min. val. loss	Best epoch	Final val. accuracy
ReLU	1	100	0.0878	14	0.9771
Sigmoid	2	100	0.0892	15	0.9768
Leaky ReLU	2	100	0.0875	6	0.9777

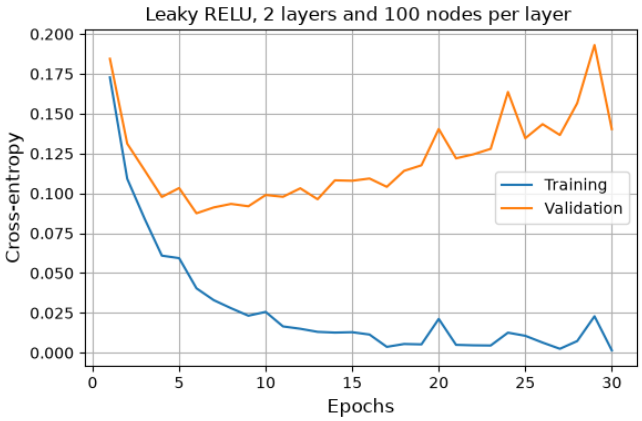


FIG. 7. Training and validation cross-entropy loss over training epochs for a neural network with two hidden layers, each with 100 nodes and Leaky ReLU activations.

Table IX shows the effect of the regularization parameter λ for the L_2 penalty. The lowest validation loss, 0.0795, was obtained with $\lambda = 0.001$ after 22 epochs. Figures 8 and 9 show the training and validation cross entropy as a function of epoch. Without regularization ($\lambda = 0$), the training loss decreased to below 0.01, while the validation loss reached its minimum after 6 epochs and then increased to between 0.12 and 0.19. With $\lambda = 0.0001$, the validation loss stayed between about 0.09 and 0.11 after its minimum. With $\lambda = 0.001$, the training loss leveled off at about 0.045 and the validation loss decreased until about epoch 22 before it increased slightly. With $\lambda = 0.01$, both the training and the validation loss stayed above 0.2.

TABLE IX. Effect of the regularization parameter λ for the L_2 penalty on MNIST classification, for a network with two hidden layers of 100 nodes, Leaky ReLU, Adam and $\gamma = 0.001$, trained for 30 epochs. The best epoch is the epoch with the lowest validation loss. The accuracy is measured after the last epoch.

λ	Min. val. loss	Best epoch	Final val. accuracy
0	0.0875	6	0.9777
0.0001	0.0870	7	0.9778
0.001	0.0795	22	0.9728
0.01	0.2112	24	0.9462

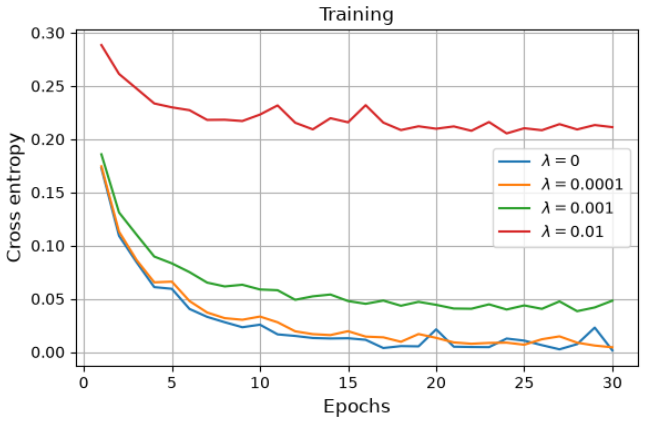


FIG. 8. Training cross-entropy loss over training epochs for a neural network with two hidden layers, each with 100 nodes and Leaky ReLU activations. Each curve corresponds to a different L_2 regularization strength λ.

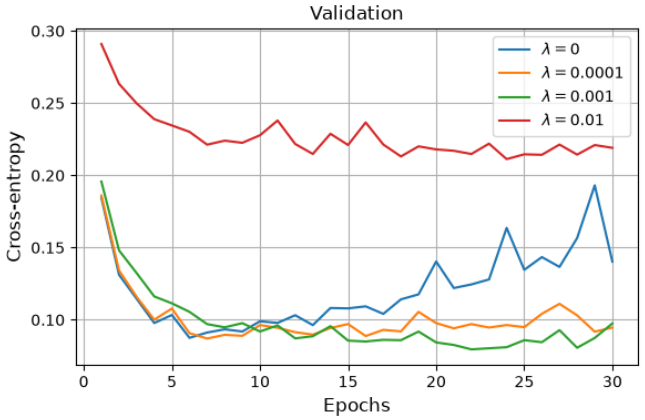


FIG. 9. Validation cross-entropy loss over training epochs for a neural network with two hidden layers, each with 100 nodes and Leaky ReLU activations. Each curve corresponds to a different L_2 regularization strength λ.

The selected network that obtained the lowest validation cross-entropy loss, had two hidden layers of 100 nodes with Leaky ReLU and was trained with Adam, $\gamma = 0.001$ and $\lambda = 0.001$. The model was retrained using the selected parameters. Table X lists the validation and test results. The network reached a validation accuracy of 0.9778 and a test accuracy of 0.9777, with a test cross entropy of 0.0693. The validation accuracy is higher than the 0.9728 in Table IX, since the final network was trained for 22 epochs instead of 30. Logistic regression from Scikit-Learn[7] reached a test accuracy of 0.9253, with a test cross entropy of 0.2784.

TABLE X. Validation and test performance of the selected network, trained for 22 epochs and test performance of logistic regression from Scikit-Learn.

Model and data set	Accuracy	Cross entropy
Network, validation	0.9778	0.0795
Network, test	0.9777	0.0693
Logistic regression, test	0.9253	0.2784

Figure 10 shows the confusion matrix for the test set. The diagonal entries count correct predictions for each digit, while the off-diagonal entries show which digits were confused. The most frequent errors were a 4 predicted as a 9 (16 times) and a 7 predicted as a 2 (12 times). The digit 6 had the lowest fraction of correct predictions, 96.6 %, followed by 9 (96.8 %) and 7 (97.0 %). The most confused pairs have similar shapes in handwriting.

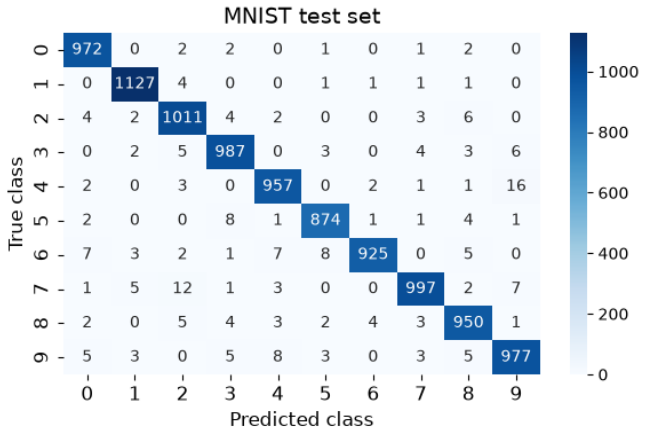


FIG. 10. Confusion matrix for the selected network on the MNIST test set. Rows represent true labels and columns represent predicted labels. Diagonal entries show the number of correctly classified images for each digit.

V. DISCUSSION

A. Learning rate and comparison with OLS

The results presented in Sec. IV A suggest that sensitivity to the learning rate may depend on the network architecture. The network with two hidden layers of 100 nodes appeared more sensitive to high learning rates than the network with one hidden layer of 50 nodes. This may reflect a more difficult optimization problem, where large parameter updates can make training unstable. However, for RMSprop and Adam, the validation MSE was nearly constant over a broad range of learning rates. The exact location of the minimum is therefore uncertain and differences in the selected learning rates should not be overinterpreted. With only two architectures and a single seed, these results do not establish that larger networks generally require smaller learning rates.

Both networks achieved lower test MSEs than the degree 8 OLS model from Project 1: 0.0151 and 0.0128, compared with 0.0187 for OLS. These results suggest that the networks provided more accurate predictions in this experiment, with the larger network achieving the lowest test MSE. However, the small test set of 20 points and the use of a single seed limit the strength of this comparison. The observed differences alone do not establish whether the improvement would persist across different data splits and initializations.

One possible explanation for the lower errors is the flexibility of the neural networks. OLS fits coefficients in a fixed polynomial basis, which limits the functions it can represent at a given degree. Neural networks can instead learn nonlinear features that may help represent both the central peak and the flatter regions of the Runge function. This flexibility also introduces additional choices and computational costs. Training depends on the learning rate, initialization and a non-convex optimization problem and each configuration was trained for 10000 epochs, whereas OLS has a closed-form solution. Greater network complexity can also increase the ability to fit noise, so the lower test MSE of the larger network does not imply that increasing complexity will generally improve performance.

The added Gaussian noise provides a useful reference for interpreting the errors. With $\sigma = 0.1$, the noise variance is $\sigma^2 = 0.01$, which is the expected MSE of a model that predicts the underlying noise-free Runge function exactly when evaluated against independently sampled noisy observations. The network errors are relatively close to this reference, suggesting that the models capture much of the underlying function. However, an MSE calculated from only 20 observations can vary considerably, so proximity to the noise variance does not by itself demonstrate that further improvements would be negligible.

To assess the robustness of these findings, future work could repeat each configuration using several random seeds and report the mean and standard deviation of the resulting errors. Repeating the data splits would help assess sensitivity to the sampled training and test data, while varying the initialization would help assess training variability. For models evaluated on the same test points, the standard error of the mean difference in squared errors could also help quantify the uncertainty in their performance difference. Finally, k-fold cross-validation on the 80 development points could reduce dependence on a single validation split while retaining the separate test set for final evaluation.

B. Testing different activation functions and depths of the neural network

The architectures tested were relatively small. The heatmaps in Figures 3 and 4 show indications of overfitting for the deeper ReLU and Leaky ReLU activation functions. The general trend in the figures is that the training MSE generally decreases when the number of nodes increased, while the validation MSE generally increases. This suggests that these larger networks started to include the noise from the data set resulting in overfitting.

With 15 nodes per layer, the sigmoid activation function showed high errors and increased for both training and validation MSE. This may originate from difficulty during optimization and caused by vanishing gradients. The derivative of the sigmoid function never exceeds $\frac{1}{4}$ and approaches zero for very large positive and negative inputs. In backpropagation these values are multiplied backwards and these cause the gradients in the early layers to become very small. Adding more layers therefore makes training harder. The MSE values are consistent with this, but do not show it directly.

The heatmaps show that the high MSE values observed for sigmoid are less pronounced with 50 and 100 nodes per layer. Increasing the number of nodes may provide greater capacity to approximate the Runge function, but it does not necessarily resolve the vanishing gradient problem.

These results suggest that increasing the network width can improve sigmoid performance within the configurations considered. The sigmoid function combined with 2 hidden layers and 50 nodes per layer achieved the lowest validation MSE among the sigmoid networks tested.

C. Testing different norms

The L_1 and L_2 searches selected the same network architecture and activation function. In both searches, the lowest validation MSE was obtained with $\lambda = 0$, which corresponds to an unregularized network. Within the configurations tested, adding either penalty did not improve validation performance.

Both searches selected sigmoid as the preferred activation function. Since both sigmoid and the Runge function are smooth functions, it could be a reason why it is preferred to approximate the function. The fact that both searches obtained lowest loss with no regularization, suggests that the unregularized network already generalized well enough without penalizing the weights.

The L_1 and L_2 penalties correspond to those used in Lasso and Ridge regression, respectively. The polynomial models from Project 1 both selected a small, non-zero penalty, while the neural network used no regularization. The neural network achieved a lower test MSE than both polynomial models. This improved performance might be caused by the neural network's ability to learn non-linear representations through its hidden layers, whereas Ridge and Lasso fit coefficients in a fixed polynomial basis.

However, the comparison is only a weak indication, as the selected polynomial was found by minimizing test MSE after training on 80 data points, while neural network hyperparameters were selected using a validation set of 20 data points before evaluating the test MSE. Even though the results suggest that the neural network performed better on this data set than Ridge and Lasso, the different procedures and the small test MSE difference limit the conclusions that can be made.

D. Classification analysis using neural networks

Adam achieved a lower minimum validation cross-entropy than RMSprop at every learning rate tested (Table VII). This supports its selection for the subsequent experiments, although the comparison is limited to the tested settings.

The architecture search selected two hidden layers with 100 neurons per layer and Leaky ReLU activations. However, the validation cross-entropy reached its minimum at epoch 6 and then increased while the training cross-entropy continued to decrease (Figure 7). This indicates overfitting and shows that further training did not improve generalization.

Adding L_2 regularization with $\lambda = 10^{-3}$ reduced the minimum validation cross-entropy and delayed its occurrence to epoch 22. The relatively stable validation loss suggests that this penalty helped limit overfitting (Figures 8 and 9). In contrast, $\lambda = 10^{-2}$ caused both losses to plateau at comparatively high values, suggesting underfitting due to an overly strong penalty.

The selected network was retrained for 22 epochs and achieved a test accuracy of 97.77% and a test cross-entropy of 0.0693. Logistic regression achieved 92.53% and 0.2784, respectively, on the same test set. The neural network therefore improved accuracy by 5.24 percentage points. Its nonlinear hidden layers allow it to learn more complex decision boundaries than logistic regression, which may explain its better performance on MNIST.

E. Critical evaluation of the various algorithms

For Runge regression, the selected FFNN achieved a test MSE of 0.0141, compared with 0.01766 and 0.01745 for Ridge and Lasso in Project 1, respectively. Although the FFNN achieved a slightly lower test MSE, using a validation set to select degree and penalty for Ridge and Lasso would make the comparison more reliable. Cross-validation and repeated experiments with different random seeds would further improve reliability in comparing neural network performance with linear regression.

On the same MNIST test set, the FFNN achieved an accuracy of 97.77%, compared with 92.53% for logistic regression. This provides stronger evidence of a performance advantage for the FFNN in the classification task.

The FFNNs hidden layers can learn nonlinear representations, but this flexibility requires additional hyperparameter tuning, including architecture, activation functions and regularization strength. Ridge and Lasso use a fixed polynomial basis, making their fitted models easier to inspect, while logistic regression provides a simpler classification baseline with linear decision boundaries in the original pixel space.

Finally, L_2 regularization improved validation performance on MNIST, whereas the selected Runge network used no weight penalty. Regularization should therefore be selected based on validation performance for the task at hand.

VI. CONCLUSION

The aim of this project was to implement an FFNN for regression and classification and to analyze the effects of the optimizer, architecture, learning rate and regularization.

For regression on the Runge function, the selected network reached a test MSE of 0.0128, compared with 0.0187 for OLS regression with polynomial degree 8 from Project 1. Adam gave the lowest validation MSE for both architectures. The best learning rate was 0.46 for one hidden layer and 0.0046 for two hidden layers. The validation errors were close to the noise level $\sigma^2 = 0.01$.

In the search over activation functions and architecture, sigmoid with two layers and 50 nodes gave the lowest validation MSE of 0.011. Some of the deeper ReLU and Leaky ReLU networks showed signs of overfitting. Increasing the network size did not consistently improve generalization. Both the L_1 and L_2 searches selected $\lambda = 0$ and neither penalty improved the validation performance within the tested configurations. The selected network produced a lower test MSE than Ridge and Lasso regression, but differences in methods limit the comparison.

For MNIST, Adam reached a lower validation loss than RMSprop at all tested learning rates. The selected network, with two hidden layers of 100 nodes, Leaky ReLU and L_2 regularization with $\lambda = 0.001$, reached a test accuracy of 97.77 %, compared with 92.53 % for logistic regression. The most frequent error was a 4 predicted as a 9. Unlike for the Runge function, regularization lowered the validation loss.

For the smooth one-dimensional Runge function, the network improved only moderately on polynomial regression, at a considerably higher cost in training and hyperparameter tuning. For MNIST, where the relevant features are not known in advance, the network clearly outperformed logistic regression. The results for the Runge function are based on a test set of 20 points and a single seed, so small differences between the models should be interpreted with care.

[1] GeeksforGeeks, "Introduction to neural networks," (2026), accessed: 2026-10-19.
[2] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning* (MIT Press, 2016).
[3] S. H. Osenbroch, T. E. J. Madsen, and W. F. Naper, "Project 1: Regression analysis, resampling methods and gradient descent," (2026), FYS-STK3155/4155, University of Oslo.
[4] F. Lee, "What is a neural network?" (2026), accessed: 2026-10-19.
[5] M. Hjorth-Jensen, "Machine learning and the physical sciences: From discriminative methods to generative methods and reinforcement learning," (2026), lecture notes, September 23, 2026.
[6] M. A. Nielsen, *Neural Networks and Deep Learning* (Determination Press, 2015).
[7] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research **12**, 2825–2830 (2011).
[8] Y. LeCun, C. Cortes, and C. J. C. Burges, "MNIST handwritten digit database," (2010).

DECLARATION OF LLM USE

Tools used

- Claude Opus 5.5 (claude.ai, October–November 2026)
- ChatGPT (chatgpt.com, October–November 2026)
- GitHub Copilot, in Visual Studio Code (October–November 2026)

Text writing and editing

Section	Level	Notes
Abstract	2	Language corrections and feedback on clarity. Checking number consistency with the rest of the text.
Introduction	2	Suggestions for rephrasing. Content checked and edited by the authors.
Theory	2	Suggestions for rephrasing. Help with tables and figures. Content checked and edited by the authors.
Methods	2	Suggestions for rephrasing. Content checked and edited by the authors.
Results	2	Suggestions for rephrasing. Help with tables and figures. Checking number consistency with the rest of the text. Content checked and edited by the authors.
Discussion	2	Language corrections and feedback on clarity. Help with removing repetition and irrelevant information. Checking number consistency with the rest of the text.
Conclusion	1	Language corrections and feedback on clarity.
Appendix	1	Language corrections and feedback on clarity. Checking number consistency with the rest of the text.

Code generation and assistance

File / notebook	Level	Description
`nn.py`	2	Assistance with debugging and explaining the code. All results checked by the authors.
`optimizers.py`	2	Assistance with debugging and explaining the code. All results checked by the authors.
`data.py`	2	Assistance with debugging and explaining the code. All results checked by the authors.
`part_b_learningrate.ipynb`	2	Assistance with debugging, explaining the code, creating plots and tables. All results checked by the authors.
`part_cd_activations_regularization.ipynb`	2	Assistance with debugging, explaining the code, creating plots and tables. All results checked by the authors.
`part_d_Lassocomparison.ipynb`	2	Assistance with debugging and explaining the code. All results checked by the authors.
`part_e_classification_an.ipynb`	2	Assistance with debugging, explaining the code, creating plots and tables. All results checked by the authors.

Appendix A: Derivative of the sigmoid function

The derivative of the sigmoid function enters every backpropagation step through a sigmoid layer and it is the reason for the vanishing gradient problem discussed in Sec. II C. This appendix shows that the derivative can be written in terms of the function value itself, which makes it cheap to compute from the stored activations.

With $f(z) = (1 + e^{-z})^{-1}$ and $u = 1 + e^{-z}$, the chain rule gives

$$f'(z) = \frac{d}{du}\left(u^{-1}\right)\frac{du}{dz} = -u^{-2}\left(-e^{-z}\right) = \frac{e^{-z}}{(1+e^{-z})^2}. \tag{A1}$$

Splitting the fraction gives

$$f'(z) = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} = f(z)\big(1 - f(z)\big), \tag{A2}$$

since $e^{-z}/(1+e^{-z}) = 1 - f(z)$. The product $f(z)(1-f(z))$ is largest when $f(z) = 1/2$, that is at $z = 0$, where it equals $1/4$. This is the upper bound used in the discussion of vanishing gradients.

Appendix B: Softmax with multiclass cross entropy

The MNIST classification uses a softmax output with the multiclass cross entropy. Each softmax output depends on all the inputs to the output layer, so its derivative is a full matrix and cannot be applied element-wise as for the other activation functions. This appendix shows that the combined derivative of the softmax and the cross entropy reduces to a simple expression, which is what the implementation uses.

Let $S_i = \sum_{r=1}^{K} e^{z_{ir}}$, so that $\hat{y}_{ik} = e^{z_{ik}}/S_i$. The quotient rule gives

$$\frac{\partial \hat{y}_{ik}}{\partial z_{ij}} = \frac{\delta_{kj}\, e^{z_{ik}} S_i - e^{z_{ik}} e^{z_{ij}}}{S_i^2} = \hat{y}_{ik}\left(\delta_{kj} - \hat{y}_{ij}\right), \tag{B1}$$

where δ_{kj} is the Kronecker delta. The derivative of the cross entropy with respect to the prediction is $\partial C_{\mathrm{CE}}/\partial \hat{y}_{ik} = -y_{ik}/(n\,\hat{y}_{ik})$. Summing over the outputs gives

$$\begin{aligned}\frac{\partial C_{\mathrm{CE}}}{\partial z_{ij}} &= \sum_{k=1}^{K}\left(-\frac{y_{ik}}{n\,\hat{y}_{ik}}\right)\hat{y}_{ik}\left(\delta_{kj} - \hat{y}_{ij}\right) \\ &= -\frac{1}{n}\left(y_{ij} - \hat{y}_{ij}\sum_{k=1}^{K} y_{ik}\right) = \frac{\hat{y}_{ij} - y_{ij}}{n},\end{aligned} \tag{B2}$$

where the last step uses $\sum_k y_{ik} = 1$ for a one-hot target. The output error is therefore the prediction minus the target and the full softmax derivative never has to be computed.